

자연어처리1 7주차

↗ 큰 계획	🔄 02. 자연어처리1
🔍 과목	자연어 처리1 1주차,자연어처리1 3주차,자연어처리1 4주차,자연어처리1 4주차 과제,자연어처리1 팀플 (제출용),자연어처리1 팀플 주제 발표 논의,자연어처리1 5주차,자연어처리1 중간고사,자연어처리1 팀플, 추가할 것,자연어처리1 6주차,자연어처리1 2주차,자연어처리1 7주차,자연어처리1 9주차,자연어처리1 10주차,자연어처리1 11주차,자연어처리1 12주차,자연어처리1 13주차,자연어처리1 14주차,자연어처리1 기말고사,유튜브 스크립트 코드 (GPT),유튜브 스크립트 데이터셋(캐글),최인엽 교수님께 여쭙보기
🔍 공부	02. 자연어처리1
📋 Test prep	Before starting
📅 계획 날짜	@2025년 4월 16일
☰ 카테고리	
📌 우선순위	1:today
☑ 완료	☐
Σ yesterday	☒
🔍 PP	0.55
↗ Sub-item	📄 <u>시험정리</u>
☀ 완료도	Not started
🔗 git	https://github.com/WegraLee/deep-learning-from-scratch-2
☰ with?	
☰ 어디서 할건데	

이름	학번
이민우	202204249
류동우	202204046

Github

<https://github.com/minuum/NLP1>

train.py

```
import os
import pickle
from common.trainer import Trainer
from common.optimizer import Adam

# 현재 작업 디렉토리를 기준으로 ROOT 경로 설정
ROOT = os.getenv('NLP1_ROOT', os.path.abspath(os.getcwd()))

def src_path(*paths):
    return os.path.join(ROOT, 'source', 'deep-learning-from-scratch-2', *paths)

def common_path(*paths):
    return os.path.join(ROOT, 'common', *paths)

# 동적으로 경로 추가
import sys
sys.path.append(os.path.join(ROOT, 'source', 'deep-learning-from-scratch-2'))
sys.path.append(os.path.join(ROOT, 'common'))

# 경로가 제대로 설정되었는지 확인 (디버깅용)
print(f"ROOT: {ROOT}")
print(f"Source path: {src_path()}")
print(f"Common path: {common_path()}")
```

```

from ch04.cbow import CBOW
from ch04.skip_gram import SkipGram
from common.util import create_contexts_target, to_cpu, to_gpu
import ptb
import config
import numpy as np

# 하이퍼파라미터 설정
window_size = 5
hidden_size = 100
batch_size = 100
max_epoch = 10

# 데이터 읽기
corpus, word_to_id, id_to_word = ptb.load_data('train')
vocab_size = len(word_to_id)

contexts, target = create_contexts_target(corpus, window_size)
if config.GPU:
    contexts, target = to_gpu(contexts), to_gpu(target)

# 모델/옵티마이저/트레이너 정의
model = CBOW(vocab_size, hidden_size, window_size, corpus)
optimizer = Adam()
trainer = Trainer(model, optimizer)

# 학습 시작
trainer.fit(contexts, target, max_epoch, batch_size)
trainer.plot()

# 나중에 사용할 수 있도록 파라미터 저장
word_vecs = model.word_vecs
if config.GPU:
    word_vecs = to_cpu(word_vecs)

params = {}
params['word_vecs'] = word_vecs.astype(np.float16)
params['word_to_id'] = word_to_id

```

```
params['id_to_word'] = id_to_word
```

```
pkl_file = 'source/deep-learning-from-scratch-2/ch04/cbow_params.pkl'  
# or 'skipgram_params.pkl'  
with open(pkl_file, 'wb') as f:  
    pickle.dump(params, f, -1)
```

1. 데이터를 로드하고 context, target를 지정해서 CBOW와 Adam을 사용하여 학습시작.
2. 학습된 단어 벡터를 저장하고 그에 따른 id도 저장.
3. 매개변수들을 serialize하고 가장 높은 pickle protocol을 사용하여 저장.

CBOW.py

그럼 CBOW의 알고리즘은?

```
class CBOW:  
    def __init__(self, vocab_size, hidden_size, window_size, corpus):  
        V, H = vocab_size, hidden_size  
  
        # 가중치 초기화  
        W_in = 0.01 * np.random.randn(V, H).astype('f')  
        W_out = 0.01 * np.random.randn(V, H).astype('f')  
  
        # 계층 생성  
        self.in_layers = []  
        for i in range(2 * window_size):  
            layer = Embedding(W_in) # Embedding 계층 사용  
            self.in_layers.append(layer)  
        self.ns_loss = NegativeSamplingLoss(W_out, corpus, power=0.75, sample  
  
        # 모든 가중치와 기울기를 배열에 모은다.  
        layers = self.in_layers + [self.ns_loss]  
        self.params, self.grads = [], []  
        for layer in layers:  
            self.params += layer.params  
            self.grads += layer.grads
```

```
# 인스턴스 변수에 단어의 분산 표현을 저장한다.  
self.word_vecs = W_in
```

```
def forward(self, contexts, target):  
    h = 0  
    for i, layer in enumerate(self.in_layers):  
        h += layer.forward(contexts[:, i])  
    h *= 1 / len(self.in_layers)  
    loss = self.ns_loss.forward(h, target)  
    return loss
```

```
def backward(self, dout=1):  
    dout = self.ns_loss.backward(dout)  
    dout *= 1 / len(self.in_layers)  
    for layer in self.in_layers:  
        layer.backward(dout)  
    return None
```

Embedding() 계층을 통과해 in_layers에 추가하고

NegativeSamplingLoss()에서 5개의 sample들과 지수를 0.75로 지정하고 이를 통해 sampling된 손실을 구한다.

손실과 가중치 및 기울기를 layers에 저장

CBOW의 forward()

`enumerate()` : 컬렉션에서 (index, element) 쌍을 반환한다.

`forward(context[:, i])` : 배열의 슬라이싱으로 모든 행과 i 번째 열을 선택해서 순전파 진행 및 h에 누적 합.

`h *= 1 / len(self.layers)` 모든 문맥 단어의 임베딩을 평균하여 문맥 벡터를 생성한다.

이 벡터와 타겟 단어를 부정적 샘플링 손실 함수에 전달한다.

CBOW의 backward()

`dout` 를 손실 함수의 backward에 매개변수로 사용하여 그 결과를 저장.

저장한 값의 평균을 내고,

그 값을 매개변수로 사용하여 각 계층의 backward로 계산된다.

```

class NegativeSamplingLoss:
    def __init__(self, W, corpus, power=0.75, sample_size=5):
        self.sample_size = sample_size
        self.sampler = UnigramSampler(corpus, power, sample_size)
        self.loss_layers = [SigmoidWithLoss() for _ in range(sample_size + 1)]
        self.embed_dot_layers = [EmbeddingDot(W) for _ in range(sample_size + 1)]

        self.params, self.grads = [], []
        for layer in self.embed_dot_layers:
            self.params += layer.params
            self.grads += layer.grads

    def forward(self, h, target):
        batch_size = target.shape[0]
        negative_sample = self.sampler.get_negative_sample(target)

        # 긍정적 예 순전파
        score = self.embed_dot_layers[0].forward(h, target)
        correct_label = np.ones(batch_size, dtype=np.int32)
        loss = self.loss_layers[0].forward(score, correct_label)

        # 부정적 예 순전파
        negative_label = np.zeros(batch_size, dtype=np.int32)
        for i in range(self.sample_size):
            negative_target = negative_sample[:, i]
            score = self.embed_dot_layers[1 + i].forward(h, negative_target)
            loss += self.loss_layers[1 + i].forward(score, negative_label)

        return loss

    def backward(self, dout=1):
        dh = 0
        for l0, l1 in zip(self.loss_layers, self.embed_dot_layers):
            dscore = l0.backward(dout)
            dh += l1.backward(dscore)

        return dh

```

모든 단어의 손실을 계산하는 대신 긍정적인 예제와 소수의 부정적인 예제에 대해서만 계산하여 학습 효율성을 높인다.

`w`: 출력 단어 임베딩 행렬 (가중치)

`corpus`: 단어 ID로 구성된 말뭉치

`power`: 단어 빈도를 조정하는 지수 (통상 0.75 사용)

`sample_size`: 각 훈련 스텝에서 생성할 부정적 샘플 수

`[SigmoidWithLoss() for _ in range(sample_size + 1)]` 는 C++로

```
std::vector<SigmoidWithLoss> loss_layers;
for (int i = 0; i < sample_size + 1; ++i) {
    loss_layers.push_back(SigmoidWithLoss());
}
```

`SigmoidWithLoss()` 객체를 `sample_size+1` 만큼 `loss_layers` 에 넣는다.

NegativeSamplingLoss의 forward()

긍정적 예 순전파

`target.shape[0]` 은 NumPy 배열 `target` 의 첫 번째 차원(행 수)를 가져온다.

`np.ones(batch_size, dtype=np.int32)` : 모든 요소가 1인 배열 생성

`score`를 입력으로 하고 라벨(긍정적 예니까 1)은

`correct_label` 사용하여 `forward()` 진행한다.

부정적 예 순전파

`negative_sample[:, i]` 는 2D 배열 `negative_sample` 의 모든 행에서 `i`번째 열만 선택함.

아래 C++ 코드와 유사

```
std::vector<SigmoidWithLoss> loss_layers;
for (int i = 0; i < sample_size + 1; ++i) {
    loss_layers.push_back(SigmoidWithLoss());
}
```

`zip()` 는 두 리스트의 요소들을 쌍으로 묶는다.

아래 C++과 유사

```

for (size_t i = 0; i < loss_layers.size(); ++i) {
    auto& l0 = loss_layers[i];
    auto& l1 = embed_dot_layers[i];
    // 이후 코드..
}

```

dout : 상위 계층에서 흘러온 기울기 (기본적으로 1)

dscore : 점수에 대한 기울기

dh : 은닉층 활성화(맥락 벡터)에 대한 기울기, 이것이 다시 이전 계층으로 전파됨

이건 그냥 궁금해서 찾아봄

부정적 샘플링은 다음 손실 함수를 최적화합니다:

$$L = -\log \sigma(v_{target}^T \cdot h) - \sum_{i=1}^k \log \sigma(-v_{neg_i}^T \cdot h)$$

여기서:

- σ 는 시그모이드 함수: $\sigma(x) = \frac{1}{1+e^{-x}}$
- v_{target} 은 타겟 단어의 임베딩 벡터
- v_{neg_i} 는 i 번째 부정적 샘플의 임베딩 벡터
- h 는 맥락 벡터
- k 는 부정적 샘플 수 (**sample_size**)

이 손실 함수는 타겟 단어와 맥락의 유사도를 높이고, 부정적 샘플들과 맥락의 유사도를 낮추는 방향으로 최적화됩니다.

```

def get_negative_sample(self, target):
    batch_size = target.shape[0]

    if not GPU:
        negative_sample = np.zeros((batch_size, self.sample_size), dtype=np.i

    for i in range(batch_size):
        p = self.word_p.copy()
        target_idx = target[i]
        p[target_idx] = 0

```



```

        p /= p.sum()
        negative_sample[i, :] = np.random.choice(self.vocab_size, size=self.
else:
    # GPU(cupy) 로 계산할 때는 속도를 우선한다.
    # 부정적 예에 타겟이 포함될 수 있다.
    negative_sample = np.random.choice(self.vocab_size, size=(batch_size,
        replace=True, p=self.word_p)

return negative_sample

```

- `target.shape[0]` : 배열의 첫 번째 차원 크기를 가져온다(배치 크기).
- `np.zeros((batch_size, self.sample_size), dtype=np.int32)` : 지정된 모양과 데이터 타입의 0으로 채워진 2D 배열을 생성한다.
- `np.random.choice(self.vocab_size, size=self.sample_size, replace=False, p=p)` : 확률 분포 `p` 에 따라 `range(vocab_size)` 에서 `sample_size` 요소를 무작위로 선택한다. `replace=False` 는 비복원 추출을 의미함
- CPU 모드에서는 각 타겟 단어에 대해 부정적 샘플을 생성한다.
 - 타겟 단어의 확률을 0으로 설정
 - 확률을 다시 정규화
 - 비복원 추출로 `sample_size` 단어 샘플링

```

class EmbeddingDot:
    def __init__(self, W):
        self.embed = Embedding(W)
        self.params = self.embed.params
        self.grads = self.embed.grads
        self.cache = None

    def forward(self, h, idx):
        target_W = self.embed.forward(idx)
        out = np.sum(target_W * h, axis=1)

        self.cache = (h, target_W)
        return out

```

```
def backward(self, dout):
    h, target_W = self.cache
    dout = dout.reshape(dout.shape[0], 1)

    dtarget_W = dout * h
    self.embed.backward(dtarget_W)
    dh = dout * target_W
    return dh
```

문맥 벡터와 단어 임베딩 간의 내적을 구현하는 클래스

W : 단어 임베딩 행렬

EmbeddingDot의 forward

은닉 벡터 **h** 와 타겟 단어의 임베딩 간의 내적을 계산한다.

역전파를 위해 입력을 캐시한다.

`np.sum(target_W * h, axis=1)` : axis=1을 따라 합계를 계산한다. axis 매개변수는 어느 축으로 축소할지 지정한다.

`self.cache = (h, target_W)` : h와 target_W를 포함하는 튜플을 생성하여 나중에 역전파에서 사용하기 위해 저장한다.

CBOW 학습 시연

```
params['word_to_id'] = word_to_id
params['id_to_word'] = id_to_word

pkl_file = 'source/deep-learning-from-scratch-2/ch04/cbow_params.pkl' # or 'skipgram_params.pkl'
with open(pkl_file, 'wb') as f:
    pickle.dump(params, f, -1)
```

21m 21.8s Python

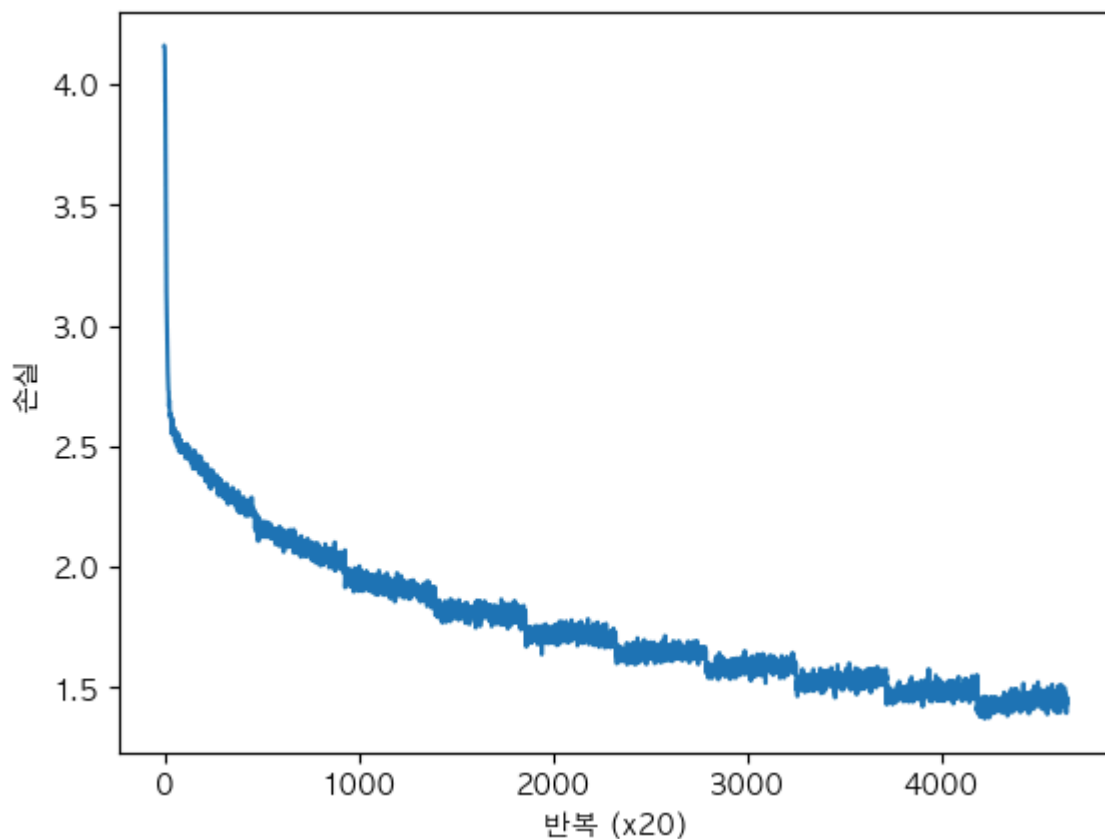
ROOT: /Users/mino/Desktop/4-1/NLP1
Source path: /Users/mino/Desktop/4-1/NLP1/source/deep-learning-from-scratch-2
Common path: /Users/mino/Desktop/4-1/NLP1/common
Downloading ptb.train.txt ...
Done

에폭	반복	시간	손실
1	1	0[s]	4.16
1	21	0[s]	4.16
1	41	0[s]	4.15
1	61	1[s]	4.12
1	81	1[s]	4.04
1	101	1[s]	3.91
1	121	2[s]	3.77
1	141	2[s]	3.63
1	161	3[s]	3.48
1	181	3[s]	3.37
1	201	3[s]	3.25
1	221	4[s]	3.14
1	241	4[s]	3.09
1	261	5[s]	3.02

```

| 에폭 1 | 반복 81 | 9295 | 시간 1[s] | 손실 4.04
| 에폭 1 | 반복 101 | 9295 | 시간 1[s] | 손실 3.91
| 에폭 1 | 반복 121 | 9295 | 시간 2[s] | 손실 3.77
| 에폭 1 | 반복 141 | 9295 | 시간 2[s] | 손실 3.63
| 에폭 1 | 반복 161 | 9295 | 시간 3[s] | 손실 3.48
| 에폭 1 | 반복 181 | 9295 | 시간 3[s] | 손실 3.37
| 에폭 1 | 반복 201 | 9295 | 시간 3[s] | 손실 3.25
| 에폭 1 | 반복 221 | 9295 | 시간 4[s] | 손실 3.14
| 에폭 1 | 반복 241 | 9295 | 시간 4[s] | 손실 3.09
| 에폭 1 | 반복 261 | 9295 | 시간 5[s] | 손실 3.02
| 에폭 1 | 반복 281 | 9295 | 시간 5[s] | 손실 2.95
| 에폭 1 | 반복 301 | 9295 | 시간 5[s] | 손실 2.91
| 에폭 1 | 반복 321 | 9295 | 시간 6[s] | 손실 2.89
| 에폭 1 | 반복 341 | 9295 | 시간 6[s] | 손실 2.84
| 에폭 1 | 반복 361 | 9295 | 시간 6[s] | 손실 2.81
| 에폭 1 | 반복 381 | 9295 | 시간 7[s] | 손실 2.78
...
| 에폭 7 | 반복 501 | 9295 | 시간 1157[s] | 손실 1.55
| 에폭 7 | 반복 521 | 9295 | 시간 1157[s] | 손실 1.59
| 에폭 7 | 반복 541 | 9295 | 시간 1158[s] | 손실 1.57
| 에폭 7 | 반복 561 | 9295 | 시간 1158[s] | 손실 1.58

```



Test with Trained CBOW Params

```

import sys
sys.path.append('.') # 상위 디렉터리 모듈 불러오기 위한 설정
from common.util import most_similar, analogy
import pickle

# 저장된 학습 파라미터 파일 불러오기

```

```
pkl_file = 'source/deep-learning-from-scratch-2/ch04/cbow_params.pkl'
# 또는 'skipgram_params.pkl'
```

```
with open(pkl_file, 'rb') as f:
    params = pickle.load(f)
    word_vecs = params['word_vecs']
    word_to_id = params['word_to_id']
    id_to_word = params['id_to_word']
```

```
# 가장 비슷한 단어(top-N) 찾기
```

```
querys = ['you', 'year', 'car', 'toyota']
```

```
for query in querys:
```

```
    most_similar(query, word_to_id, id_to_word, word_vecs, top=5)
```

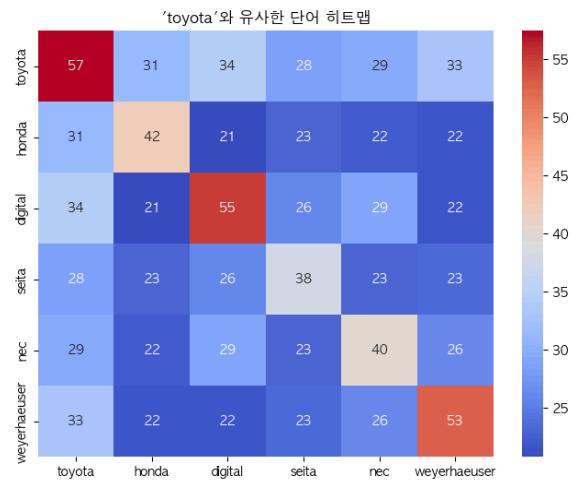
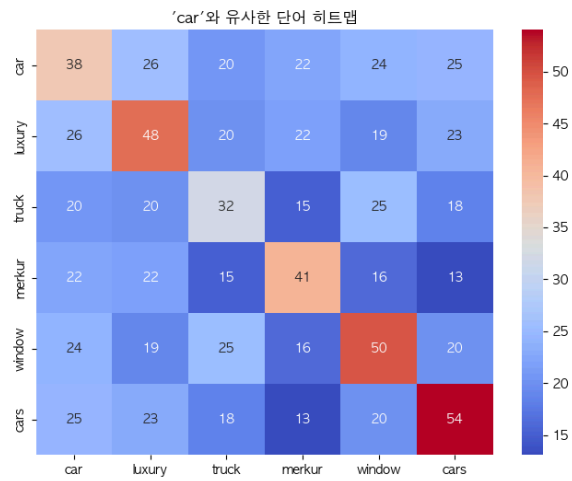
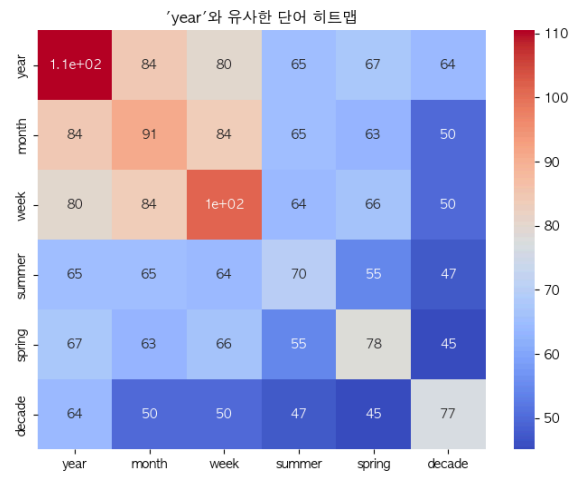
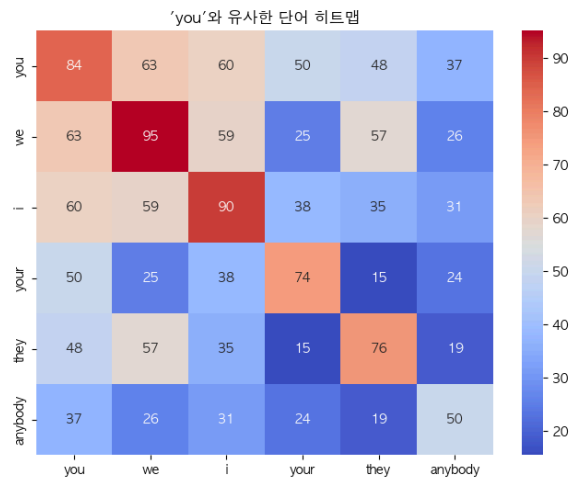
```
[query] you
we: 0.6103515625
someone: 0.59130859375
i: 0.55419921875
something: 0.48974609375
anyone: 0.47314453125
```

```
[query] year
month: 0.71875
week: 0.65234375
spring: 0.62744140625
summer: 0.6259765625
decade: 0.603515625
```

```
[query] car
luxury: 0.497314453125
arabia: 0.47802734375
auto: 0.47119140625
disk-drive: 0.450927734375
travel: 0.4091796875
```

```
[query] toyota
ford: 0.55078125
instrumentation: 0.509765625
mazda: 0.49365234375
bethlehem: 0.47509765625
nissan: 0.474853515625
```

[+ Code](#) [+ Markdown](#)



Query	대표 클러스터	패턴
you	인칭 대명사	대명사들의 유사성 잘 포착
year	시간 표현	다양한 시간 단위 (연, 월, 주, 계절)
car	차량/브랜드/부품	단수-복수, 브랜드 포함
toyota	자동차 브랜드 + 기업명	자동차보단 기업 이미지에 더 가까움